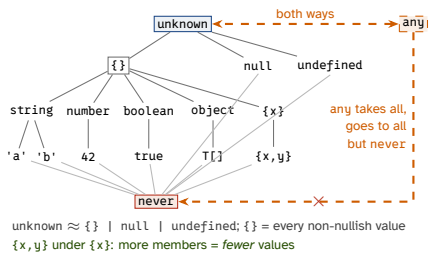
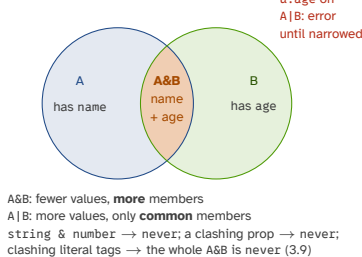


In one line: A type is a **set of values**; assignability is **subset**, judged by **shape**, not by name. A union must be **narrowed** along control flow before use. Types are **erased**; enums, namespaces with code and parameter properties are TypeScript syntax that emits JavaScript. Soundness is traded away on purpose in known places: any, as, method parameters, readonly aliasing.

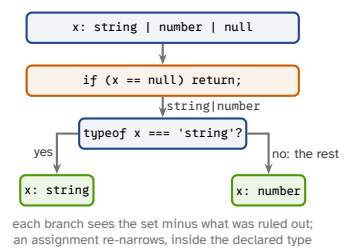
1 types are sets (up = bigger)



2 values, not members



3 control-flow narrowing



Assignability is structural

- x is compatible with y if y has at least the members of x; names never matter. Classes compare instance members only; a private/protected member must come from the same declaration — the nominal escape.
- Excess properties:** only a fresh literal put straight into a typed slot — *Object literal may only specify known properties, and 'y' does not exist in type 'P'*. (2353). Via a variable, as or an index signature it passes.
- Weak type** (all optional, 2.4): even via a variable one property must match — *Type '{ payload: string; ... }' has no properties in common with type 'Options'*. (2559)
- object (2.2) = non-primitives; "object is not Object".

Top, bottom, special

type	takes $\rightarrow$ to	what to remember
unknown	all $\rightarrow$ unknown, any	no member, call or arithmetic until narrowed (==, !== allowed); T unknown = unknown, T & unknown = T; keyof unknown = never; truthy $\rightarrow$ { } (4.8)
any	all $\rightarrow$ all but never	checking off; members of any are any, so it spreads
never	none $\rightarrow$ all	empty set: T never = T; what is left when every case is handled; (): never = cannot return (throws, loops) $\neq$ undefined: a () $\Rightarrow$ void slot takes () $\Rightarrow$ number (result ignored); a function declared : void may not return a value 5 and "" too — not "an empty object"
void	undefined	
{ }	all but nullish	

Literals, widening, as const

const s = 'a' is 'a', let s = 'a' is string; properties widen too: const req = { method: 'GET' } gives method: string, so passing it to a 'GET' | 'POST' parameter fails — *Argument of type 'string' is not assignable to parameter of type 'GET' | 'POST'*. as const: no widening, readonly properties and tuples ([10, 20] as const: readonly [10, 20]); literals only; a referenced array inside stays mutable.

annotation · satisfies · as

```
const r1: Record<string, string> = { home: '/' };
r1.hmoe; // compiles: the annotation IS the type
const r2 = { home: '/' } satisfies Record<string, string>;
r2.hmoe; // error: inferred { home: string } is kept
const r3 = { } as Record<'home', string>; // compiles: a lie
const n = 'hi' as number; // 2352 "... may be a mistake"
const m = 'hi' as unknown as number; // forces anything
```

as: only to a more or less specific type. x!: non-null, unchecked.

Functions · readonly · optional

- Fewer parameters fit (forEach(x $\Rightarrow$...)); returns are covariant; optional $\leftrightarrow$ required interchangeable; a rest = endless optionals.
- Parameters compare **bivariantly** (unsound) unless strictFunctionTypes: then contravariantly — but **method syntax** m(x: T): void stays bivariant "to ensure ... Array<T> ... relate covariantly". Strict callback: onX: (e: E) $\Rightarrow$ void.
- 5.8 checks each branch of a returned c ? a : b: in (s: string): URL, c ? anyMap.get(s) : s gives *Type 'string' is not assignable to type 'URL'*.
- readonly is ignored by assignability: { readonly x } goes into { x } and is written through the alias. Arrays differ: *The type 'readonly string[]' is 'readonly' and cannot be assigned to the mutable type 'string[]'*. (4104).
- a?: T may be absent, a: T | undefined must be present; exactOptionalPropertyTypes bans writing a: undefined.

Merging · interface vs type

interfaces of one name merge: a shared property must keep its type (2717), methods become overloads, the later block's first: a type cannot reopen (*Duplicate identifier*). A namespace merges into a class, function or enum, a class never into a class or variable; declare module '.x' only patches existing names. Only type names unions and tuples; extends reports a clashing property, & silently makes it never.

Narrowing — control-flow analysis

- typeof: "string" "number" "bigint" "boolean" "symbol" "undefined" "object" "function" — typeof null is "object". Truthiness drops 0 NaN "" 0n null undefined: if (count) loses 0. == null matches undefined too.
- "k" in x keeps members with k (optional k: both branches); a key no member lists gives x & Record<"k", unknown> (4.9). instanceof: prototype chain.
- An assignment narrows within the declared type: let x = c ? 10 : 'hi'; x = true — *Type 'boolean' is not assignable to type 'string | number'*.
- Also tracked: a test kept in a const (4.4); const { kind, payload } = a (4.6); switch (true) (5.3); a closure made after the last assignment (5.4; none if a closure assigns); obj[key] with both unchanged (5.5). A test truthy by its very form is an error (5.6): *This kind of expression is always truthy. — if (!e/)*.

Discriminated unions · guards

```
type Shape =
  | { kind: 'circle'; r: number }
  | { kind: 'rect'; w: number; h: number };
function area(s: Shape): number {
  switch (s.kind) {
    case 'circle': return Math.PI * s.r ** 2;
    case 'rect': return s.w * s.h;
    default: const gone: never = s; return gone;
  }
}
const ns = [1, undefined, 2].filter(x => x !== undefined);
// number[]: the predicate x is number is inferred
```

- Tag = a property every member has, typed by literals. A new kind fails gone: never (Type '{ kind: "tri"; ... }' is not assignable to type 'never'), s satisfies never (1360), or — with no default under a declared return type — *Function lacks ending return statement and return type does not include 'undefined'*. (2366).
- A predicate body is **trusted, not checked**: $\therefore$ x is string $\Rightarrow$ true compiles. Inferred only with no return annotation, one return, an unmutated parameter, true $\Rightarrow$ T and false $\Rightarrow$ not T: so not for x $\Rightarrow$!!x on number | undefined (0).
- asserts c / asserts v is T narrow after the call (the function throws otherwise). The callee needs an explicitly typed name: const check = (c: unknown): asserts c => ...; check(v) — *Assertions require every name in the call target to be declared with an explicit type annotation.* (2775)

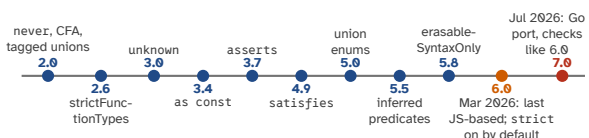
Enums as types

- Numeric:** any number value fits, an out-of-range literal does not (5.0: 1 into enum E { A = 0, B = 2 } errors; two enums never mix).
- String:** nominal — const d: Dir = 'UP' gives *Type "UP" is not assignable to type 'Dir'*.
- An enum is a union of its members; each member is a type that narrows (type Primary = Color.Red | Color.Blue). Emit, const enum, erasability and the as const alternative: ts-config-and-practice

Index signatures · tuples

- Keys: string, number, symbol, template patterns ([k: `data-\${string}`]): unknown, 4.4), unions of them. Under [k: string]: number, name: string is an error; a number index must fit the string one.
- [number, number, number?] has length: 2 | 3; a rest goes anywhere ([...boolean[], string]); readonly [string, number] refuses pair[0] = ... — *Cannot assign to '0' because it is a read-only property.*

Timeline



Related: ts-generics-advanced (variance, conditional and mapped types) · ts-config-and-practice (strict, brands, parsing at the edge) · js-types-coercion-modules · Typed contracts across iOS, Android, web & backend · RN testing + TypeScript · Kotlin type system & generics — variance, reified, nullability